

xstockstrat — CI, Local Stack, and Cloud Infrastructure

A walkthrough of how 15 services build, test, and ship: GitHub Actions for CI, Docker Compose for local development, and DigitalOcean App Platform for dev and prod deploys.

Video Outline (the spine)

Time	Beat	What to show
0:00 – 0:20	What you're looking at. One repo, three runtime contexts: local (compose), dev (DO), prod (DO). Same images, different specs.	Title card. Diagram: repo → CI → registry → DO dev & prod.
0:20 – 0:55	Local stack: docker-compose. TimescaleDB, OTel collector, 13 services, nginx. One <code>docker compose up</code> and the platform is live on <code>localhost</code> .	<code>docker compose up -d</code> , <code>docker compose ps</code> listing healthy services.
0:55 – 1:30	The proto-first build. Stubs are generated from <code>.proto</code> files via a single codegen container. CI's <code>proto-freshness</code> job blocks any drift between committed stubs and what <code>buf generate</code> would produce.	<code>./scripts/buf-gen.sh</code> running, then the CI job catching a stale stub.
1:30 – 2:15	CI: path-filtered, per-service jobs. A <code>paths-filter</code> step detects which services changed; only their lint/test/coverage jobs run. Coverage thresholds enforced per language.	CI YAML showing matrix jobs. GitHub Actions UI showing only changed services running.
2:15 – 2:55	Branch protection + required checks. <code>main</code> and <code>main-dev</code> block direct pushes. <code>CI / Proto lint and breaking check</code> is a required check. A PR cannot merge with a red status.	Settings page showing branch protection rules.
2:55 – 3:40	Deploy on merge: dev (paper) vs prod (live). Push to <code>main-dev</code> → <code>Deploy dev</code> workflow → DO CR image push → DO App Platform update with <code>.do/app.dev.yaml</code> . Push to <code>main</code> → same path with <code>.do/app.yaml</code> .	Workflow run page side-by-side with the two app specs.
3:40 – 4:20	DO App Platform. App spec is checked into the repo. Each service is a separate component with <code>instance_size</code> , <code>instance_count</code> , env vars, and a DO CR image reference. Managed TimescaleDB attached as <code>databases:</code>	<code>.do/app.yaml</code> showing one service block, the managed DB block.

4:20 – 4:55	Secrets & registry. Secrets injected from GitHub Actions on every deploy. DigitalOcean Container Registry (DOCR) stores images per-SHA + a <code>latest / latest-dev</code> tag. Old images rotated by retention policy.	DOCR repo browser, GitHub secrets settings (blurred values).
4:55 – 5:30	Observability hooks. OTEL collector in compose pipes traces locally; production uses OTLP env vars pointing at Grafana Cloud. <code>OTEL_ENABLED=true</code> everywhere.	Grafana Cloud trace view.
5:30 – 6:00	Safety. <code>buf breaking</code> blocks contract regressions. Secret scanning (trufflehog + gitleaks) runs on every PR. <code>db-migrator</code> <code>PRE_DEPLOY</code> job runs migrations before service restarts on every push.	Failing PR with secret scan annotation; successful migration log.
6:00 – 6:30	Outro. All infra config is in the repo. Reproducible from a fresh DO account in under an hour using <code>/digitalocean-setup</code>	<code>/digitalocean-setup</code> interactive skill running.

The video can be tightened to 3–5 min by collapsing sections 0:55–1:30 and 4:20–4:55 into condensed segments, or extended toward 6:00 for a full infra tour.

Section 1 — Three Runtime Contexts, One Repo

The same code runs in three places:

Context	Compose / orchestrator	Trading mode	Branch	Spec file
Local development	<code>docker-compose.yml</code> (host Docker)	paper (hardcoded)	any	<code>docker-compose.yml</code>
Dev (DigitalOcean)	DO App Platform	paper	<code>main-dev</code>	<code>.do/app.dev.yaml</code>
Production (DigitalOcean)	DO App Platform	live	<code>main</code>	<code>.do/app.yaml</code>

The trading mode environment variables (`TRADING_MODE`, `ALPACA_PAPER`, `ALPACA_BASE_URL`) are set at the spec level, not in the config service. A live deploy cannot accidentally paper-trade; a dev deploy cannot accidentally live-trade.

Section 2 — Local Stack: docker-compose.yml

`docker-compose.yml` is the single source of truth for local development. One command brings up the entire platform on `localhost`:

```

cp .env.example .env          # fill in ALPACA_API_KEY, ALPACA_API_SECRET, JWT_SECRET
./scripts/bootstrap.sh        # checks Docker, generates proto stubs
docker compose up -d          # starts everything
docker compose ps              # all services should be Up / healthy

```

Services running:

Component	Container	Purpose
timescaledb	TimescaleDB	Postgres + time-series hypertables, schema-per-service
db-migrator	golang-migrate	Runs all <code>services/*/migrations/*.up.sql</code> on startup
otel-collector	OTel Collector	Receives OTLP, forwards to Grafana Cloud (or stdout in dev)
xstockstrat-config	Node.js	<code>WatchConfig</code> stream — must come up first
xstockstrat-ledger	Node.js	Append-only event store
xstockstrat-identity	Node.js	JWT issuance and verification
xstockstrat-notify	Node.js	Alert streaming
xstockstrat-marketdata	Go	Alpaca feed, OHLCV storage
xstockstrat-portfolio	Go	Position tracking, P&L
xstockstrat-trading	Go	Order lifecycle
xstockstrat-indicators	Python	Formula sandbox
xstockstrat-ingest	Python	Signal normalization
xstockstrat-analysis	Python	Strategy backtesting
xstockstrat-trader	Next.js	UI :3000
xstockstrat-insights	Next.js	UI :3001
xstockstrat-config-ui	Next.js	UI :3002
nginx	Nginx	Reverse proxy → all three UIs :80

Compose patterns

- **Healthchecks on every service.** A service is only marked `healthy` when its readiness probe passes (`/health` for HTTP services, gRPC health-check for gRPC).
- **depends_on with condition: service_healthy** enforces startup order. The config service comes up first; every other service waits for it.

- **WAIT_FOR entrypoint** in each service: blocks until upstream dependencies are reachable. Survives container restart ordering races.
- **Three env files:** `.env` (secrets, not committed), `.env.local` (structural, committed), `.env.fe.local` (frontend-only, committed). Compose merges all three.

Section 3 — Proto-First Build

Every service consumes generated stubs from `packages/proto/`. The single source of truth lives in `.proto` files; Go, Python, and TypeScript stubs are committed alongside them.

Codegen container

`Dockerfile.codegen` is a single container with `buf` and all three language stub generators pre-installed. Developers do not need Go, Python, or Node installed on the host to regenerate stubs:

```
./scripts/buf-gen.sh
# runs buf inside Dockerfile.codegen container
# writes Go stubs to packages/proto/gen/go/
# writes Python stubs to packages/proto/gen/python/
# writes TypeScript stubs to packages/proto/gen/ts/ (then compiles)
```

Freshness enforcement

A CI job named **"Proto stubs are up to date"** runs `./scripts/buf-gen.sh` and then `git diff --exit-code`. If the committed stubs don't match what regeneration produces, the job fails. This catches:

- A `.proto` edit committed without re-running `buf-gen`
- A `buf` version drift between local and CI
- A generated file edited by hand (forbidden — re-running `buf-gen` always wins)

Breaking change gates

Two `buf` jobs run on every PR:

- `buf lint` — style and structure checks (always required to pass).
- `buf breaking --against main` — fails on any field removal, type change, RPC removal, or other breaking change.

The combined job is named `CI / Proto lint and breaking check` and is the **single required status check** for both `main-dev` and `main` branch protection. No PR can merge with this check red.

Section 4 — CI: Path-Filtered Per-Service Jobs

`.github/workflows/ci.yml` is built around a `paths-filter` step that detects which services changed in the PR. Only those services' `lint/test/coverage` jobs run. CI on a single-service change finishes in ~5 minutes; a full proto change touching all services takes longer but is still parallelized.

The job matrix

Job	Trigger	Coverage threshold
Detect changed paths	always	—
Proto lint and breaking check	proto OR ci change	hard gate
Proto stubs are up to date	proto OR ci change	hard gate
Go lint and test ({service})	per-Go-service change	≥40%
Python lint ({service})	per-Python-service change	—
Python test and coverage ({service})	per-Python-service change	≥40% (indicators ≥50%)
Node lint ({service})	per-Node-service change	—
Node test and coverage ({service})	per-Node-service change	≥40%
Frontend E2E ({service})	per-Next.js-service change	Playwright pass
secret-scan	always	hard gate (trufflehog + gitleaks)

Per-language tooling

Language	Lint	Test	Coverage
Go 1.25	golangci-lint v2.5.0	go test -race -coverprofile	go tool cover -func ≥40%
Python 3.12	ruff check + ruff format --check	pytest --cov	≥40% (≥50% indicators)
Node.js 22	eslint via pnpm run lint	vitest --coverage	≥40%
Next.js	eslint + Playwright E2E	playwright test	golden-path scenarios

Every Go job sets `GOWORK=off` so per-service builds use the service's own `go.mod`, not the workspace.

Section 5 — Branch Protection and Merge Strategy

Two protected branches:

- `main-dev` — development trunk. All feature branches and `claude/*` branches PR here. Squash

and merge.

- **main** — production. Only PRs from `main-dev` and `hotfix/*`. **Merge commit, never squash** (squashing breaks the ancestry; `main-dev` would stay permanently "ahead" of `main`).

Both branches enforce:

Rule	Value
Required status check	CI / Proto lint and breaking check
Require branch up to date before merge	yes (strict)
Required PR reviews	1 approving review
Dismiss stale reviews on new commits	yes
Block direct pushes	yes

Configured via `./scripts/setup-branch-protection.sh` (one-time, uses `gh api`).

Section 6 — Deploy on Merge: Two Pipelines

Push to `main-dev` → Deploy dev (paper)

```
push to main-dev
├─ .github/workflows/deploy-dev.yml
│  ├── buf-push-dev    # publish proto as draft to BSR
│  ├── build-images    # build all 15 service Docker images, push to DOCR
│  └─ deploy           # doctl apps update $DO_DEV_APP_ID --spec .do/app.dev.yaml
│      └─ DO redeloys components with TRADING_MODE=paper
```

Push to `main` → Deploy prod (live)

```
push to main
├─ .github/workflows/deploy-prod.yml
│  ├── buf-push        # publish proto (final) to BSR
│  ├── build-images    # build all 15 service Docker images, push to DOCR
│  └─ deploy           # doctl apps update $DO_PROD_APP_ID --spec .do/app.yaml
│      └─ DO redeloys components with TRADING_MODE=live
```

The deploy job is a reusable workflow (`.github/workflows/deploy.yml`) parameterized by:

- `DO_APP_ID` — dev vs prod app
- `app_spec` — `.do/app.dev.yaml` **VS** `.do/app.yaml`
- `image_tag` — short SHA of the commit being deployed
- `BROKER_ACCOUNTS_ENCRYPTION_KEY` — separate dev / prod values pulled from GitHub secrets

Image registry

DigitalOcean Container Registry (DOCR). Every push gets two tags:

- The **short SHA** of the commit (`a1b2c3d`) — immutable, used for the exact deploy
- A **floating tag** (`latest-dev` or `latest`) — points at the most recent successful build for that environment

Rollback = re-deploy with a previous SHA tag. No image rebuild needed.

Section 7 — DigitalOcean App Platform Specs

`.do/app.yaml` and `.do/app.dev.yaml` are the full deploy specs, checked into the repo. Each is ~530 lines covering 15 service components, a managed Postgres + TimescaleDB cluster, and a `db-migrator` `PRE_DEPLOY` job.

What's in a spec

For each service:

```
- name: xstockstrat-trading
  image:
    registry_type: DOCR
    repository: xstockstrat-trading
    tag: latest
  instance_size_slug: apps-s-1vcpu-1gb
  instance_count: 1
  http_port: 8051
  envs:
    - key: TRADING_MODE
      value: live # paper in app.dev.yaml
    - key: ALPACA_BASE_URL
      value: https://api.alpaca.markets # paper-api.alpaca.markets in dev
    - key: DATABASE_URL
      type: SECRET
      value: ${db.CONNECTION_STRING}
    # ... other env vars
  health_check:
    http_path: /health
```

What's managed

- **Postgres + TimescaleDB cluster** declared in the `databases:` block. DO provisions and connects via `${db.CONNECTION_STRING}`.
- **PRE_DEPLOY job** (`db-migrator`) runs `scripts/db-migrate.sh` against the cluster before any service restarts. `golang-migrate` is idempotent; already-applied migrations are skipped.
- **Internal DNS** — services reach each other via `xstockstrat-<name>.<app>.svc.internal` on prod and bare service names on dev.

Secrets injection

Secrets values are set with `type: SECRET`. They are sourced from GitHub Actions secrets on every deploy via `sed` substitution in the deploy workflow. No secret values live in the repo or git history.

Section 8 — Database Migrations

	Local	DO dev	DO prod
Tool	golang-migrate	golang-migrate	golang-migrate
Trigger	<code>./scripts/db-migrate.sh</code> (manual) or <code>db-migrator</code> compose service	PRE_DEPLOY job on every push to <code>main-dev</code>	PRE_DEPLOY job on every push to <code>main</code>
Source	<code>services/*/migrations/NNN_*.up.sql</code>	same	same
Tracking	<code><schema>.schema_migrations</code>	same	same
Order	trading → portfolio → marketdata → indicators → ingest → analysis → ledger → identity → notify → config	same	same

Convention: each service owns its own schema. Migrations are `NNN_description.up.sql` + `.down.sql`. **Never edit an applied migration** — add a new numbered one instead. CI's `db-migrator` runs idempotently on every deploy.

Section 9 — Observability

OpenTelemetry SDK in every service (Go, Python, Node.js) exports OTLP. Toggle:

```
OTEL_ENABLED=true.
```

Local

`docker-compose.yml` runs an `otel-collector` container. The collector receives OTLP from all services and (optionally) forwards to Grafana Cloud if `OTEL_EXPORTER_OTLP_HEADERS` is set, otherwise just logs to stdout.

Production

Services emit OTLP directly to Grafana Cloud:

```
- key: OTEL_EXPORTER_OTLP_ENDPOINT
  value: https://otlp-gateway-prod-<region>.grafana.net/otlp
- key: OTEL_EXPORTER_OTLP_HEADERS
  type: SECRET
  value: Authorization=Basic ...
```

OTel init errors **never** block service startup. If the collector is down, the service runs without telemetry

— it does not crash-loop.

Trace propagation

`x-trace-id` is propagated through every gRPC and Connect-RPC call via a service-specific interceptor:

- Go: `internal/telemetry/` package, unary + stream interceptors
- Python: per-method header propagation
- Node.js: `AsyncLocalStorage` carries the trace ID through async chains

Section 10 — Secret Hygiene

Secret scanning runs on **every PR** as part of CI:

- **trufflehog** — pattern-based scan across the full commit history
- **gitleaks** — second-source scanner with custom rules in `.gitleaks.toml`

Both must pass. Public-repo audit script `scripts/security-audit.sh` runs the same checks locally before any sensitive change.

Secret namespaces in config:

- `secret.*` keys are never logged
- `secret.*` keys are never shipped to telemetry
- `secret.*` keys are read-only via the config UI (admins only)

GitHub Actions secrets injected at deploy time:

- `DIGITALOCEAN_ACCESS_TOKEN` (deploy auth)
- `DO_REGISTRY_NAME` (image push target)
- `DO_DEV_APP_ID` / `DO_PROD_APP_ID`
- `DEV_BROKER_ACCOUNTS_ENCRYPTION_KEY` / `PROD_BROKER_ACCOUNTS_ENCRYPTION_KEY`
- `BUF_TOKEN` (BSR proto publishing)

None of these values are in the repo. The deploy workflow reads them from the GitHub secrets API at run time.

Section 11 — First-Time Setup Skill

The whole DO infrastructure can be bootstrapped from a fresh account using the `/digitalocean-setup` skill (interactive, 9 numbered steps):

1. `doctl` auth and project setup
2. DOCR (Container Registry) creation
3. Managed PostgreSQL + TimescaleDB cluster
4. Dev App Platform app from `.do/app.dev.yaml`
- 4.5. DOCR-to-app linkage

5. Prod App Platform app from `.do/app.yaml`
6. Secrets wiring (GitHub Actions → DO)
7. GitHub Actions workflow validation
8. First deployment trigger
9. Smoke test

Each step is idempotent — re-running the skill on a partially-configured account picks up where the last run stopped. The skill prompts before any destructive action.

Section 12 — Reproducibility Checklist

What you need to recreate this infrastructure from scratch:

- [] A DigitalOcean account with billing enabled
- [] A Grafana Cloud account (free tier works) — for OTel ingestion
- [] An Alpaca account (paper for dev, live for prod) — for trading APIs
- [] A BSR account — for proto publishing (optional; CI can skip BSR push)
- [] A GitHub repo with Actions secrets configured (script + values via `/digitalocean-setup`)
- [] `gh`, `doctl`, and `docker` CLIs installed locally for the first-time setup

End-to-end time from empty DO account to live dev deploy: **~45 minutes** with `/digitalocean-setup`.

Outro

All infrastructure config — CI workflows, compose file, DO specs, migrations, secret-scanning rules — is checked into this repo. Nothing depends on a private wiki or a `~/.config` file. Pull the repo, run the setup skills, and you have the same dev + prod environment.

Repository: github.com/davcs86/xstockstrat **Setup runbooks:** [docs/setup/](#) **CI overview:** [docs/patterns/ci-overview.md](#) **Docker patterns:** [docs/patterns/docker-build.md](#)